



ECHO-TWIN PROJECT · INAF

Crack & Detachment Segmentation

User Manual — interface guide, installation instructions, keyboard shortcuts, and application workflow.

This application supports the manual and semi-automated annotation of cracks and material detachments on building facade photographs, turning raw inspection imagery into pixel-accurate, structured training data for automated defect-detection models. Developed within **Work Package 3 of the ECHO-TWIN project at INAF**, it underpins the construction of digital-twin models for structural health monitoring — producing the annotated datasets used to fine-tune deep-learning segmentation models on facade imagery acquired by drone.

Version 1.0.5

- **A*-guided tracing** — assisted, pixel-accurate crack & detachment outlining.
- **SIFT-based projection** — imports and aligns cracks across photos of the same building.
- **Automatic building grouping** — and batch shape-compatibility checks across a group.



macOS

Windows

Linux

1 Interface Overview

The user interface consists of three main sections that allow you to operate in a single desktop window:

Toolbar & Menus Located at the top. Contains drop-down menus (File, Edit, Tools, etc.) and quick-access buttons for common operations — image navigation, saving, importing. Scrolls horizontally (mouse wheel/trackpad) if the window is too narrow to show every button at once.	Workspace (Canvas) The central area where the building photograph is displayed, and where you trace the outlines of cracks and detachments.	Shortcuts Sidebar To the right of the canvas. Clickable buttons for the commands not already on the toolbar (tool selection, view toggles, calibration) — each triggers the action directly, just like a toolbar click. Scrolls vertically if the window is too short to show every button at once.	Status Bar Located at the bottom, with a scrollbar. Shows real-time console log messages, warnings, and operation status (e.g. successful PNG masks or JSON exports).
--	---	---	---

2 Installation

Pre-built, self-contained packages are available for macOS, Windows, and Linux — no separate Python installation is required on the target machine. Alternatively, each platform can also be built directly from the source code using the scripts in the `packaging` folder (`build_mac.sh` , `build_linux.sh` , `build_windows.bat`) — see the “Building from source” box under each platform below.

macOS

.dmg

1. Double-click `CrackSegmentation.dmg` in Finder to open it (or, from a terminal, run `open CrackSegmentation.dmg`).

2. A Finder window opens showing the app and a shortcut to the Applications folder. Drag **CrackSegmentation** onto the Applications shortcut.

3. Eject the disk image, then launch the app from Applications or Launchpad as usual.

Do **not** run the .dmg with `./CrackSegmentation.dmg` from a terminal — a .dmg is a disk image, not a script.

On first launch, macOS Gatekeeper may block the app because it isn’t signed with a paid Apple Developer certificate. Allow it once via **System Settings > Privacy & Security > “Open Anyway”**, or right-click the app and choose **Open**.

Building from source instead

1. Install Python 3 (the python.org installer already includes Tkinter; with Homebrew, also run `brew install python-tk`).

2. Open Terminal in the extracted source’s `packaging` folder.

3. Run `./build_mac.sh` (first time: `chmod +x build_mac.sh`).

4. Result: `packaging/dist/CrackSegmentation.app` and `packaging/dist/CrackSegmentation.dmg` .

Windows

.exe

1. Run `CrackSegmentation-Setup.exe` .

2. Follow the installer wizard (choose an install location, create a Start Menu shortcut).

3. Launch **Crack Segmentation** from the Start Menu once installation completes.

Windows SmartScreen may flag the installer as unrecognized, since it isn’t signed with a paid code-signing certificate. This is a standard warning for independently built software, not a sign of a problem — click **More info > Run anyway** to continue.

Building from source instead

1. Install Python 3, checking “Add python.exe to PATH” in the installer (or run `winget install Python.Python.3.12 --source winget`), then close and reopen the terminal.

2. Optional, for a real Setup.exe: install [Inno Setup](#) first.

3. Open Command Prompt/PowerShell in the extracted source’s `packaging` folder and run `build_windows.bat` .

4. Result: `packaging\dist\CrackSegmentation-Setup.exe` (with Inno Setup) or `packaging\dist\CrackSegmentation\CrackSegmentation.exe` standalone (without it).

For Debian/Ubuntu-based distributions, install the provided package from a terminal:

```
sudo apt install ./cracksegmentation_1.0.5_amd64.deb
```

Alternatively:

```
sudo dpkg -i ./cracksegmentation_1.0.5_amd64.deb
sudo apt-get install -f
```

After installing, launch the tool by typing `cracksegmentation` in a terminal, or find **Crack Segmentation** in your desktop’s Applications menu.

The package is unsigned, so your system may warn about installing software from an unknown source — expected for a locally built package without a GPG-signed repository behind it.

Building from source instead

- 1. Install Tkinter and venv support: `sudo apt install python3-tk python3-venv`.
- 2. Open a terminal in the extracted source’s `packaging` folder.
- 3. Run `chmod +x build_linux.sh && ./build_linux.sh` — it builds inside an isolated `.venv` it creates automatically, without touching system Python packages.
- 4. Result: `packaging/dist/cracksegmentation_<version>_amd64.deb` — install it the same way as above.

3 Tested Configurations

The application has been verified to launch and run correctly on the following real-world machines:

OS	Device	Processor	RAM	Notes
macOS Tahoe	MacBook Pro (MacBookPro17,1)	Apple M1 (8-core: 4P+4E)	8 GB	Model no. MYD92T/A
Windows 11 Home	HP Laptop 15s-fq5xxx	Intel Core i7-1255U @ 1.70 GHz	16 GB	64-bit, x64
Ubuntu 24.04.4 LTS	Victus by HP Gaming Laptop 16-r1xxx	Intel Core i7-14700HX	16 GB	NVIDIA GeForce RTX 4050 Max-Q (AD107M)

4 Main Commands & Keyboard Shortcuts

The software is heavily optimized for quick keyboard use, allowing you to switch between tools without interrupting your workflow.

Action	Shortcut	Description
Crack Tool	<code>C</code>	Activates the mode for assisted crack tracing.
Detachment Tool	<code>D</code>	Activates the mode for drawing polygons for plaster/material detachment areas.
Save / Save & Next	<code>S</code> / <code>Enter</code>	Exports JSON and masks; <code>S</code> stays on the photo, <code>Enter</code> moves to the next.
Import & Align	<code>W</code> / <code>L</code>	Projects the cracks of a previous photo of the same building (SIFT + homography, matched around the source cracks); an unreliable match imports nothing.
Cut / Join / Link	<code>1</code> / <code>2</code> / <code>3</code>	Two clicks. Cut removes the part between them; Join re-routes it; Link merges two cracks (end of one, start of the other) and saves JSON and mask at once.
Retrace Imported Cracks	<code>V</code>	Re-traces all imported cracks (A*) onto the real edges of the new photo.
Manual Snap / Translate	<code>T</code>	Allows manual, elastic repositioning of an imported crack that hasn’t aligned correctly.
Modify Crack Thickness	<code>A</code>	Varies the thickness (px/side) of crack tracts: <code>+</code> / <code>-</code> widen or narrow the mask.
Assign Building Group	<code>B</code>	Manually assigns a unique code to the current building, useful for tracking across multiple frames.

Check Compatibility (Filter)	G	Automatically removes cracks that don't match in shape and path between different photos of the same group.
Scale Calibration	K	Starts the mode to manually calibrate the measurement scale in centimeters relative to image pixels (Pixel-to-CM).
Simple Navigation	← / →	Next/previous image without saving; wheel/trackpad pans when zoomed.
Switch Mode	Sidebar / Menu	Switches between new-images and reload-segmented data sources at any time, without restarting the application.

Visual Mask Management

While tracing cracks and defects, transparency/overlay visibility can be managed to avoid obscuring the view:

- N

 Shows or hides the blue crack-thickness band.
- Space

 Hides the markers (each start marker shows the crack number);

.

 hides messages (auto after 15 s).
- J

 Opens/closes the status info panel overlay, with each crack's reliability score (texture along the line) and confidence (0-1, ridge strength against the wall);

[?]

 marks an uncertain crack.
- ?

 Shows the on-screen user manual.

5 Workflow & Application Logic

Session Start

On launch, an initial pop-up lets you choose to work on new images (photos in

Images

 without a JSON) or reload processed ones (with a JSON in

JSON files

) for corrections. Photos stay in

Images

; masks go to

Binary files

. This choice isn't final: the **Switch Mode** sidebar button/menu item lets you swap between the two data sources at any later point in the session, without restarting the application. When the last image of the queue has been saved or skipped, a dialog tells you the images are finished (and how many skipped images are still in

Images

) and asks whether to switch to the other mode or exit the application.

Automatic Import and Tracing (A* Retrace)

When a building is covered by multiple frames, the application associates them using SIFT similarity. Cracks traced in one view can be imported into another (

W

 /

L

). Since the perspective changes, press

V

 to let the A* pathfinding algorithm automatically readjust and “paste” the imported lines onto the physical edges detected in the current photo, discarding gross distortions.

Batch Crack Cleaning and Compatibility

One of the most advanced features is handled by the

compare_and_filter_cracks.py

 module (runnable via

G

 from within the GUI, or as a standalone script). It scans the exported cracks across photos of the same building; if a crack differs too much from one photo to another — measured by the sinuosity index (how “wavy” the path is relative to a straight line between its endpoints, a metric invariant to the photo's scale/distance) — it is discarded and removed from the final JSONs to keep the data clean.

6 Authors & Credits

This software was developed within the framework of the **ECHO-TWIN** project by the INAF (Istituto Nazionale di Astrofisica) Research Group.

Research Team	
Roberto Spina ^{1,2}	Code Architecture, Software Design & Lead Development
Fabio Vitello ¹	Project Coordination & Supervision
Eva Sciacca ¹	Project Coordination & Supervision
Leonardo Pelonero ¹	General Support
Salvatore Scavo ¹	General Support
¹ INAF - Catania Astrophysical Observatory ² Order of Geologists of Sicily (National Register)	

AI Disclosure

This application utilizes artificial intelligence models for certain stages of the workflow, specifically for analysis, technical content generation, and development support. The AI components do not replace scientific validation nor do they influence results in a deterministic manner: they are used as assistance tools, not as sources of truth.